

HSI 高光谱工业视觉分析流水线

本项目实现了一套完整的高光谱图像（HSI）工业分析流水线，覆盖从**数据标注**、**模型训练**到**边缘端推理与云端上报**的全流程。

- 项目部署位置：工控机 F:\weiyutao\work\ai\hsi
- 化验数据位置：二楼机器 E:\weiyutao\weiyutao_20260525_1417
- 500 个颗粒原始采集数据位置：工控机 F:\500块采样

目录

- 项目概述
- 一、YOLO 分割模型训练流水线
 - 1.1 环境准备
 - 1.2 使用 Label Studio 标注数据
 - 1.3 切分训练数据 (split_yolo_data.py)
 - 1.4 配置 data.yaml
 - 1.5 启动训练 (train_yolo.py)
- 二、边缘端推理与上报 (hsi_inference_test.py)
 - 2.1 脚本说明
 - 2.2 核心处理流程
 - 2.3 使用步骤
- 其他脚本说明

项目概述



一、YOLO 分割模型训练流水线

1.1 环境准备

```
pip install ultralytics
pip install torch torchvision # 建议安装 CUDA 版本
```

确认 GPU 可用：

```
import torch
print(torch.cuda.get_device_name(0))
```

1.2 使用 Label Studio 标注数据

Label Studio 是本项目的数据库管理与标注平台，用于维护全部训练样本。

操作步骤：

1. **导入数据**：将采集到的高光谱伪彩图（RGB 三通道 PNG/JPG）批量导入 Label Studio 项目。
2. **配置标注模板**：选择 **Polygon** 或 **Brush** 类型的分割模板，标注类别为矿石颗粒（如 **ore**）。
3. **标注**：为每张图片勾画出所有矿石颗粒的实例掩码。
4. **导出数据**：标注完成后，选择导出格式为 **YOLO with Images**（即同时导出图片和对应的 YOLO 格式 **.txt** 标签文件）。导出后会得到一个压缩包，解压后目录结构如下：

```
project-13-at-2026-xx-xx/
├─ images/           # 原始图片
├─ labels/           # 对应的 YOLO 格式标签 (.txt)
└─ classes.txt       # 类别名称文件
```

1.3 切分训练数据（split_yolo_data.py）

将 Label Studio 导出的原始数据集按比例划分为**训练集**和**验证集**。

配置脚本底部的参数区：

```
# split_yolo_data.py 底部配置区
SOURCE_DIRECTORY    = 'project-13-at-2026-06-11-08-45-5d6e60b0' # Label Studio 解
压目录名
DESTINATION_DIRECTORY = 'alsi_seg_202606111647_v5'                # 输出的 YOLO 数据
集目录名
SPLIT_RATIO         = 0.8                                         # 80% 训练，20%
验证
```

执行：

```
python split_yolo_data.py
```

执行完毕后，生成如下标准 YOLO 数据集目录结构：

```
alsi_seg_202606111647_v5/
├── images/
│   ├── train/
│   └── val/
├── labels/
│   ├── train/
│   └── val/
└── classes.txt
```

1.4 配置 data.yaml

在项目根目录（与 `train_yolo.py` 同级）创建或确认 `data.yaml`，内容示例如下：

```
path: F:/weiyutao/work/ai/hsi/alsi_seg_202606111647_v5 # 数据集根目录（绝对路径）
train: images/train
val: images/val

nc: 1 # 类别数量
names: ['ore'] # 类别名称，与 classes.txt 保持一致
```

注意： `path` 必须填写绝对路径，避免运行时路径解析错误。

1.5 启动训练 (train_yolo.py)

确认 `train_yolo.py` 中的关键参数：

参数	说明	推荐值
<code>model</code>	预训练权重路径	<code>yolov8s-seg.pt</code> （精度与速度均衡）
<code>DATA_YAML_PATH</code>	<code>data.yaml</code> 绝对路径	见上节
<code>epochs</code>	训练轮数	<code>100</code> （可根据收敛情况调整）
<code>imgsz</code>	输入图像尺寸	<code>640</code>
<code>batch</code>	批次大小	<code>16</code> （8GB 显存）
<code>patience</code>	早停轮数	<code>30</code>

参数	说明	推荐值
project / name	结果保存目录	自定义实验名称

执行训练：

```
python train_yolo.py
```

训练完成后，最优权重保存在：

```
runs/alsi_seg_202606111647_v5/alsi_seg_202606111647_v5/weights/best.pt
```

二、边缘端推理与上报（hsi_inference_test.py）

2.1 脚本说明

hsi_inference_test.py 是工控机端高光谱图像处理与自动上报脚本，采用零落盘、全内存流设计，全程不写本地临时文件。

核心功能：

- 读取本地目录下的高光谱数据（.hdr + .spe 文件对）
- 使用训练好的 YOLOv8-seg 模型对每帧图像进行滑窗实例分割
- 对每个检测到的矿石颗粒进行连通域分离，提取其完整高光谱像素矩阵
- 将 RGB 伪彩图、全局掩码图和每个颗粒的 .npy 光谱数据以 **Multipart POST** 方式上传至后端 API

与在线分选程序（industrial_vision_cpp）严格对齐：

对齐项	在线（C++ CUDA）	离线（本脚本）
波段选取	R=44, G=63, B=88	相同
归一化方式	clip(raw/gain, 0, 1)	_normalize_fixed_gain()
增益值	7000（原始 DN） / 1.0（反射率）	根据数据类型配置
水平翻转	kFlipX=1	np.flip(axis=1)
置信度阈值	0.3	conf_thresh=0.3
IOU 阈值	0.45	iou_thresh=0.45
最小颗粒面积	10 像素	min_particle_area=10

2.2 核心处理流程



2.3 使用步骤

Step 1: 修改脚本底部配置区

打开 `hsi_inference_test.py`, 找到 `if __name__ == "__main__":` 下方的配置区, 按照现场环境修改:

```

# ① 训练好的 YOLO 权重路径
YOLO_WEIGHTS =
r"F:\weiyutao\work\ai\hsi\runs\segment\runs\alsi_seg_202606111647_v5\...\best.pt"

# ② 后端 FastAPI 接口地址
API_URL = "http://10.0.26.209:9039/api/storage/upload/hsi_processed_image"

# ③ 待处理的本地高光光谱数据目录 (包含 .hdr 和 .spe 文件)
INPUT_DIR = r"F:\weiyutao\202620630_spe"

# ④ 本批次名称 (用于后台数据归档)
BATCH_NAME = "平陆一诺铝土矿20_big_num_small_202606301113"
  
```

Step 2: 确认增益参数

根据数据类型选择正确的 `yolo_gains`:

```

# 如果 .spe 存储的是【原始 DN 值】(整型, 范围较大)
processor = HSIEdgeProcessor(..., yolo_gains=(7000.0, 7000.0, 7000.0))
  
```

```
# 如果 .spe 存储的是【反射率】(float32, 中位数约 0.1)
processor = HSIEdgeProcessor(..., yolo_gains=(1.0, 1.0, 1.0))
```

Step 3: 运行脚本

```
python hsi_inference_test.py
```

正常运行时终端输出示例：

```
🤖 [初始化] 正在加载边缘计算 YOLOv8 模型...
🔗 [初始化] 目标服务器接口: http://10.0.26.209:9039/...
📁 [开始任务] 监控目录: F:\weiyutao\202620630_spe | 批次名: 平陆...

---> [开始处理] 图像: frame_001
📤 [上传中] frame_001 包含 47 个颗粒...
✅ [成功] frame_001 成功入库。耗时: 2.34秒

🔔 [批次结束] 平陆... 处理完毕。成功: 120/120
```

Step 4: 异常排查

错误信息	可能原因	解决方式
波段数 x 异常	非 204 波段文件混入	检查 INPUT_DIR 中的文件
HTTP状态码: 4xx/5xx	后端服务异常	确认 API_URL 和服务状态
YOLO model not found	权重路径错误	检查 YOLO_WEIGHTS 路径
颗粒数为 0	增益配置错误导致图像全黑	切换 yolo_gains 参数
客户端奔溃	.spe 文件缺失或损坏	确认 .hdr/.spe 文件配对完整

其他脚本说明

脚本	功能
hsi_feature_extract.py	自编码器特征提取 (204 → 5 维)
hsi_feature_inference.py	特征推理 (在线预测)
hsi_train.py	自编码器 / 回归模型训练
label_studio_json_txt.py	Label Studio JSON 转 YOLO txt 格式辅助脚本
verify_yolo_labels.py	验证 YOLO 标签文件合法性
find_beat_band.py	筛选最优光谱波段
data_build.py / hsi_data_build.py	原始数据集构建

脚本

功能

test_yolo.py

YOLO 模型本地测试